

Audio Deepfake Spoofing Detection via Spectrogram Analysis

Project Proposal for UCS503P

Submitted to:

Dr Nisha Thakur

Thapar Institute of Engineering and Technology

Submitted by:

Aayaan Dua 1024170284

Kushagra Rai 1024170277

Jasleen Kaur 1024170091

August 2026

1 Higher-order goal

...secondary framing

This project aims to improve the reliability of digital voice communication by developing a system capable of identifying AI-generated or manipulated speech. With the increasing accessibility of voice cloning and synthetic speech generation, audio recordings can be manipulated in ways that are difficult to distinguish from genuine human speech.

While the broader objective is to address the growing problem of audio-based impersonation and misinformation, the immediate engineering objective is to develop a deployable software pipeline that transforms raw audio into meaningful representations, classifies the recording using a custom-trained CNN, and provides an interpretable prediction to the user.

The project focuses on the complete data-processing pipeline:

Audio → Preprocessing → Mel-Spectrogram → CNN → Prediction → Explanation

The system will additionally maintain prediction and experiment information in an SQL database to enable historical analysis of model performance under different audio conditions.

2 Time-to-value

...primary heuristic

The project will be developed incrementally so that a functional audio classification pipeline can be demonstrated early and progressively enhanced.

- We will first establish the basic audio ingestion and preprocessing pipeline and verify that audio files can be converted into consistent Mel-spectrogram representations.
- We will then train an initial CNN model on real and synthetic speech samples and establish a baseline classification performance.
- Subsequent iterations will introduce robustness testing using noise and compression-based degradation.

- Explainability using Grad-CAM will be added to provide visual evidence for the model's predictions.
- SQL-based logging and the web interface will be integrated progressively rather than being treated as separate components.
- Each iteration will be evaluated using measurable metrics such as accuracy, precision, recall, F1score, and inference time.

Success is therefore defined not only by producing a working classifier, but by progressively building a complete and measurable audio-forensics pipeline.

3 Problem Statement

The rapid development of AI-based voice cloning and synthetic speech technologies has made it increasingly difficult to determine whether an audio recording is genuine or artificially generated.

Traditional users generally have no reliable way to inspect an audio recording beyond listening to it. This becomes particularly problematic when synthetic voices are used for impersonation, misinformation, fraud, or other forms of social engineering.

The major engineering challenges addressed by this project include:

- **Human indistinguishability:** Modern synthetic voices can reproduce human-like speech characteristics, making manual identification difficult.
- **Complex audio representation:** Raw audio waveforms contain large amounts of information and are not directly suitable for a simple image-based CNN pipeline.
- **Lack of interpretability:** A classification system that only produces “Real” or “Fake” does not explain why the decision was made.
- **Real-world degradation:** Audio shared through communication platforms may be compressed, noisy, or otherwise degraded compared with clean training samples.
- **Short audio samples:** Potential scam or impersonation recordings may consist of only a few seconds of speech.
- **Historical analysis:** A detection system should be able to record predictions and experimental conditions so that changes in performance can be studied over time.

Why this matters now (contextual relevance): The increasing availability of generative voice technologies makes audio authenticity an increasingly relevant engineering and security problem. A practical detection system therefore needs to consider not only classification accuracy on clean datasets but also interpretability and performance under realistic audio conditions.

4 Proposed Solution

...Software Proposition

4.1 Overview

We propose a web-based “Audio Deepfake Spoofing Detection System” consisting of the following components:

- **Audio upload interface:** Users upload supported audio recordings for analysis.

- **Audio preprocessing pipeline:** Raw recordings are normalized, resampled, and prepared for feature extraction.
- **Mel-spectrogram generation:** Audio waveforms are transformed into Mel-spectrograms representing frequency information over time.
- **Custom CNN classifier:** A CNN designed and trained by the team classifies the resulting spectrogram representation as real or synthetic.
- **Confidence estimation:** The system reports the model's prediction along with its confidence score.
- **Explainable AI module:** Grad-CAM is used to highlight regions of the spectrogram that contributed to the model's prediction.
- **Robustness analysis:** Additional degraded versions of audio are generated to evaluate performance under noise and compression.
- **SQL telemetry:** Prediction results, audio metadata, model information, and experimental conditions are stored for historical analysis.

4.2 Core workflow

1. A user uploads an audio recording through the web interface.
2. The backend validates the audio format and performs preprocessing such as resampling and normalization.
3. The processed waveform is converted into a Mel-spectrogram using Python audio-processing libraries.
4. The generated spectrogram is passed to the custom CNN.
5. The CNN produces a classification and confidence score.
6. Grad-CAM generates a visual heatmap identifying regions of the spectrogram that influenced the prediction.
7. The prediction, confidence, processing information, and relevant metadata are stored in the SQL database.
8. The user receives the classification and corresponding explanation through the interface.

For short recordings, the system can additionally divide the audio into smaller overlapping windows and evaluate each segment before aggregating the results.

4.3 Operational constraints for an educational setting

- The system will initially support commonly used audio formats such as WAV and MP3.
- Model training will be performed offline, while the deployed application will primarily perform inference.
- The system will use a manageable dataset appropriate for the available computational resources.
- The project will focus on binary classification between genuine and synthetic speech rather than attempting to identify every possible voice-generation system.
- Results will be presented as model predictions rather than absolute proof of audio authenticity.

- The system will be designed so that additional datasets and generation methods can be incorporated in future iterations.

5 Solution Approach

...Engineering focus

The project combines audio signal processing, machine learning, explainable AI, and database-backed application development.

5.1 Audio ingestion and preprocessing

...data preparation

The backend will process uploaded recordings using Python-based audio-processing libraries.

The preprocessing stage will include:

- Loading the audio waveform.
- Converting recordings to a consistent sampling rate.
- Normalizing amplitude.
- Removing or reducing silent sections where appropriate.
- Converting stereo recordings to a consistent representation when required.
- Dividing longer recordings into suitable analysis windows.

The purpose of this stage is to ensure that recordings with different technical characteristics can be converted into a consistent representation before model inference.

5.2 Spectrogram transformation

...audio-to-image pipeline

The central transformation in the project is the conversion of audio into a visual representation.

The waveform will be transformed into a Mel-spectrogram, which represents the distribution of frequency energy across time using the Mel frequency scale.

This creates the primary input to the CNN:

Raw waveform → Frequency representation → Mel-spectrogram image

This transformation allows the project to apply convolutional neural network techniques to audio-frequency patterns rather than directly processing raw waveform samples.

The spectrogram generation process will be implemented programmatically so that every uploaded recording passes through the same feature-extraction pipeline.

5.3 Custom CNN architecture

...classification

Rather than relying on a pre-trained deepfake detection API, the project will develop and train a custom CNN using a framework such as TensorFlow/Keras.

The CNN will consist of convolutional and pooling layers followed by fully connected classification layers.

The training process will involve:

- Dataset preparation.

- Training/validation/test splitting.
 - CNN training.
 - Hyperparameter experimentation.
 - Validation during training.
 - Evaluation on unseen recordings.
- Comparison of different experimental conditions.

The initial objective is binary classification:

Real Audio / Synthetic Audio

Model performance will be evaluated using multiple classification metrics rather than accuracy alone.

5.4 Explainable Visual Forensics

...XAI

A major feature of the proposed system is the inclusion of model explainability.

Grad-CAM will be applied to the CNN to generate a heatmap over the Mel-spectrogram. The heatmap will indicate the areas that contributed most strongly to the model's classification.

Therefore, the system will provide:

Prediction + Confidence + Visual Explanation

This is intended to make the output more informative than a simple binary classifier and allow users to inspect which time-frequency regions were influential in the model's decision.

5.5 Real-world degradation pipeline

...robustness

To evaluate whether the system remains useful outside clean laboratory conditions, controlled audio degradation will be introduced during experimentation.

Examples include:

- Background noise.
- Audio compression.
- Reduced audio quality.
- Changes in amplitude.
- Telephone-like filtering.

The model's performance on clean and degraded recordings will then be compared.

This allows us to investigate:

How does audio degradation affect deepfake detection performance?

Rather than assuming that the model is robust, robustness will be treated as an experimentally measurable property.

5.6 Short-audio analysis

...micro-sample detection

The project will investigate the effect of reducing the amount of available speech.

Longer recordings can be divided into overlapping windows, allowing the CNN to make predictions on smaller audio segments.

For example:

10-second recording → Window 1 → Window 2 → Window 3 → Window 4

The individual predictions can then be combined to produce an overall result.

Experiments will compare different audio durations to determine how classification performance changes as less speech becomes available.

5.7 SQL-backed telemetry

...historical analysis

The SQL database will store structured information associated with model predictions and experiments.

Potential fields include:

- Audio/sample ID.
- File metadata.
- Dataset/source information.
- Real/fake ground-truth label where available.
- Predicted class.
- Prediction confidence.
- Model version.
- Audio duration.
- Processing conditions.
- Degradation type.
- Prediction timestamp.

This enables historical analysis of model performance and allows the team to compare results across different model versions and audio conditions.

5.8 CI/CD and fast delivery enabler

Continuous Integration and Deployment (CI/CD) will be used to ensure the pipeline remains stable during iterative model updates:

- Automatically build and run tests on the audio-preprocessing logic whenever code is changed.
- Produce repeatable deployment artifacts so the frontend and database can be safely pushed to staging.

- Enable safe, frequent releases of improved CNN models without breaking the web interface or SQL logging system.

6 Evaluation Criterion

...measurable and attributable

The project will define measurable evaluation criteria so that the effectiveness of the system can be objectively assessed.

6.1 Primary evaluation metrics

The primary classification metrics will be:

- **Accuracy & F1-score:**
 - **Target:** > 85% on the held-out test set.
 - **Attribution:** Measured via the automated classification script against the ground-truth labels in the dataset.
- **Inference Time (End-to-End Latency):**
 - **Target:** < 3 seconds from audio upload to visual heatmap rendering.
 - **Attribution:** Measured from SQL database timestamps (submission time vs. prediction logged time).
- **System Reliability:**
 - **Target:** > 99% uptime of the web interface during the testing window.

A confusion matrix will additionally be used to examine false-positive and false-negative behavior.

6.2 Secondary evaluation metrics

The project will evaluate:

- **Clean vs. degraded performance:** Compare model performance before and after introducing noise/compression.
- **Audio-duration performance:** Compare performance for different input durations.
Inference time: Measure the time required to process an uploaded recording.
- **Explainability:** Verify that Grad-CAM successfully produces meaningful spectrogram visualizations for model predictions.

- **Model consistency:** Compare performance across different model versions and experimental configurations.

The evaluation will be based on a held-out test set that is not used during model training.

6.3 Experimental validation plan

...high-level

The validation process will include controlled experiments across multiple conditions.

1. Train and evaluate the baseline CNN on the selected real/synthetic audio dataset.
2. Record baseline accuracy, precision, recall, and F1-score.
3. Introduce controlled noise and compression into test samples.
4. Measure the resulting performance change.
5. Evaluate recordings of different durations.
6. Generate Grad-CAM visualizations for representative predictions.
7. Store experimental results and prediction metadata in the SQL database.
8. Compare model versions and identify conditions under which the model performs poorly.
9. Recruit 5-10 pilot users to interact with the web frontend. Users will upload a mix of raw audio files to test the system's ingestion stability, verify that Grad-CAM heatmaps render correctly on the client side, and confirm that all metadata is accurately captured in the SQL database during live operations.

This approach allows the project to identify both the strengths and limitations of the proposed detector.

7 Scalability

...with foresight

The proposed architecture should support expansion in both dataset size and functionality.

1. Scalable (theoretically):

The system can support larger datasets through:

- Separate frontend and backend components.
- Batch processing during model training.
- Structured dataset organization.
- SQL indexing for frequently queried prediction records.
- Modular preprocessing and inference pipelines.
- Versioned machine-learning models.

2. Operationally deployable (practically):

The application can be deployed using:

- A web frontend.
- A Python-based backend API.
- A relational SQL database.
- Object/file storage for uploaded audio and generated spectrograms.

- Containerized or platform-based deployment.

The model and preprocessing pipeline will be kept modular so that additional datasets, audio-generation methods, and model architectures can be introduced without redesigning the entire application.

8 Engine availability heuristic

A mature ecosystem of tools and libraries is available for implementing the proposed system.

The project can use:

- Python for the core processing pipeline.
- Librosa for audio loading and feature extraction.
- NumPy for numerical operations.
- TensorFlow/Keras for CNN development and training.
- Matplotlib for spectrogram visualization.
- Grad-CAM-compatible tooling for model explainability.
- SQL/MySQL or PostgreSQL for structured telemetry storage.
- FastAPI/Flask for backend API development.
- HTML/CSS/JavaScript or a suitable frontend framework for the user interface.
- Git/GitHub for version control and collaborative development.
- CI/CD tooling for automated testing and deployment.

We will use these mature components to focus on the audio-processing pipeline, model experimentation, system integration, and evaluation rather than building basic infrastructure from scratch.

9 Project scope and deliverables

...iteration-friendly

9.1 Initial deliverable

...first iteration

The first iteration will establish the core machine-learning pipeline:

- Audio dataset collection and organization.
- Audio preprocessing pipeline.
- Mel-spectrogram generation.
- Initial custom CNN architecture.
- Training and validation pipeline.
- Baseline real/fake classification.
- Accuracy, precision, recall, and F1-score evaluation.
- Basic SQL schema for prediction records.
- Initial backend endpoint for audio analysis.

The first iteration should therefore demonstrate the complete path from:

Audio → Spectrogram → CNN → Prediction

9.2 Subsequent deliverables

Later iterations will add:

- Web-based audio upload interface.
- Confidence score visualization.
- Grad-CAM spectrogram heatmaps.
- Audio degradation and robustness experiments.
- Sliding-window short-audio analysis.
- Expanded SQL telemetry and historical analysis.
- Model-version tracking.
- Performance dashboard.
- Improved preprocessing and model tuning.
- Automated testing and deployment.

The final system will combine these components into a single usable prototype.

10 Risks and mitigations

- **Data availability and quality — Risk:** The quality and diversity of real and synthetic audio samples may affect model performance. Mitigation: use carefully separated training, validation, and test sets and document dataset sources and characteristics.
- **Dataset bias — Risk:** The model may learn speaker, recording-device, or dataset-specific characteristics instead of genuine synthetic-speech artifacts. Mitigation: use diverse speakers and recording conditions where possible and evaluate the model on samples from conditions not represented identically in training.
- **Overfitting — Risk:** A CNN trained on a limited dataset may perform well on training data but poorly on unseen samples. Mitigation: use validation data, augmentation, regularization, and a held-out test set.
- **Audio degradation — Risk:** Compression and background noise may significantly reduce classification performance. Mitigation: treat degradation as an explicit experimental condition and investigate augmentation strategies to improve robustness.
- **Explainability limitations — Risk:** A Grad-CAM heatmap indicates influential regions but does not prove that those regions represent a specific physical artifact. Mitigation: present Grad-CAM as an interpretability aid rather than definitive forensic evidence.
- **Computational requirements — Risk:** CNN training and repeated spectrogram generation may require significant computational resources. Mitigation: use controlled dataset sizes initially, optimize preprocessing, and perform training experiments incrementally.

- **False predictions — Risk:** A detector may incorrectly classify genuine or synthetic recordings. Mitigation: report confidence and standard classification metrics, and clearly communicate that the system provides a model-based assessment rather than absolute proof of authenticity.
- **Evaluation measurement ambiguity** - Risk: Latency measurements could be skewed by slow internet uploads rather than model inefficiency. Mitigation: instrument timestamps at the exact moment backend processing begins and ends, separating network delay from inference time.
- **Operational issues in pilot** - Risk: The web interface may crash if multiple users upload audio concurrently. Mitigation: use staging environments early and rely on CI/CD to ensure repeatable, stable deployments before the final demonstration.

11 Summary

This project proposes a full-stack Audio Deepfake Spoofing Detection System that combines audio signal processing, machine learning, explainable AI, and relational data management.

The central engineering pipeline transforms raw audio into Mel-spectrograms and uses a custom-trained CNN to distinguish between genuine and synthetic speech. The system extends beyond basic classification by investigating performance under audio degradation and different recording durations, while Grad-CAM provides a visual interpretation of the model's predictions.

An SQL database will maintain prediction and experiment telemetry, enabling historical analysis of model behavior across different conditions and model versions.

The project therefore emphasizes not only the development of a working AI classifier, but also data transformation, measurable evaluation, robustness testing, explainability, and deployable software architecture.

The final prototype will demonstrate the complete workflow:

**Audio Upload → Preprocessing → Mel-Spectrogram → Custom CNN → Real/Fake Prediction →
Confidence + Grad-CAM Explanation → SQL Logging**

This makes the project a practical engineering system for investigating the detection of synthetic speech while providing clear opportunities for experimentation, evaluation, and future expansion.